RaceDay

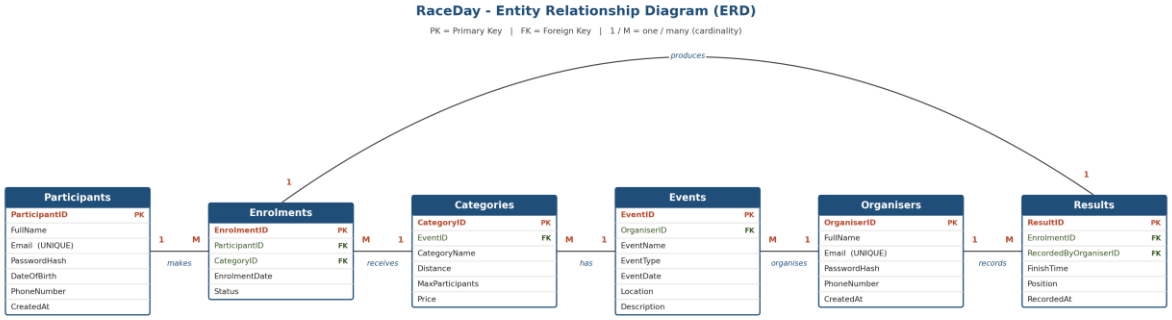
Portfolio of Evidence – Part 1
System Planning and Database

Prepared by: SIYETHEMBA XULU
ST: 10475058
Programme: Software Development, Rosebank College
Module: RaceDay Portfolio of Evidence

This document contains the planning deliverables for Part 1: the Entity Relationship Diagram (Section A) with a full explanation of every entity and relationship, the API Endpoint Plan (Section B), and the SQL database script (Section C). No API code has been written for this part – it is planning only, as required.

Section A – Entity Relationship Diagram (ERD)

The RaceDay data model contains six entities: Organisers, Participants, Events, Categories, Enrolments and Results. The diagram below shows every entity, its attributes, and the relationships between them, with primary keys (PK), foreign keys (FK) and cardinality (1 / M) marked on each connector.



Why these six entities, and why two separate role tables

Organisers and Participants are modelled as two separate entities – not one generic "Users" table with a role column – because the two roles have different attributes (Participants have a DateOfBirth for age-category checks; Organisers don't) and different permissions throughout the system. Keeping them as distinct tables means role-based access in Part 2 can be enforced simply by which table/claim a request is authenticated against, and it avoids a table full of nullable columns that only apply to one role.

Events and Categories are also kept separate rather than merged, because one event (e.g. the Vryheid Cycle Challenge) genuinely offers several distinct entry options – different distances, capacities and prices – that participants choose between. Storing distance/price/capacity at the event level would force every category of that event to share the same values, which does not reflect how South African road events are actually structured.

Enrolments exists purely to resolve the many-to-many relationship between Participants and Categories: a participant can enter many categories over time, and a category is entered by many participants, so a direct link between the two tables isn't possible in a relational model – it has to be resolved through an associative entity. Making Enrolments its own table also gives a natural place to store enrolment-specific data (date entered, status) and a natural anchor for Results, which is where the design pays off.

Results is linked to Enrolments rather than directly to Participants or Categories, because a result only makes sense in the context of one specific entry (this participant, in this category, on this occasion) – linking straight to Participants would lose track of which category/event the time belongs to.

Entity-by-entity breakdown

1. Organisers

Represents an event organiser – the user role that owns and manages events. Kept as its own table (rather than a single generic Users table with a role flag) because organisers and participants have different attributes and different permissions throughout the system, and separating them makes the role-based access rules in Part 2 straightforward to enforce at the database level.

Attribute	Type (SQL)	Key	Notes
OrganiserID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
FullName	NVARCHAR(100)		Required.
Email	NVARCHAR(150)		UNIQUE – used for login and to stop duplicate accounts.
PasswordHash	NVARCHAR(255)		Never stores a plain-text password.

PhoneNumber	NVARCHAR(20)		Optional contact number.
CreatedAt	DATETIME		Defaults to GETDATE() when the row is inserted.

2. Participants

Represents the second user role – the runner/walker/cyclist who browses events, enrols in categories, and tracks personal results. Structurally similar to Organisers but includes DateOfBirth, which the system may need for age-category eligibility.

Attribute	Type (SQL)	Key	Notes
ParticipantID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
FullName	NVARCHAR(100)		Required.
Email	NVARCHAR(150)		UNIQUE – used for login.
PasswordHash	NVARCHAR(255)		Never stores a plain-text password.
DateOfBirth	DATE		Optional; supports age-category checks.
PhoneNumber	NVARCHAR(20)		Optional contact number.
CreatedAt	DATETIME		Defaults to GETDATE().

3. Events

A single race-day event (e.g. “Vryheid Community Fun Run”) created and owned by one organiser. This is the parent of Categories – an event is the umbrella under which multiple distance/price options are offered.

Attribute	Type (SQL)	Key	Notes
EventID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
OrganiserID	INT	**FK**	References Organisers.OrganiserID – identifies who owns/manages this event.
EventName	NVARCHAR(150)		Required.
EventType	NVARCHAR(50)		Running / Walking / Cycling.
EventDate	DATETIME		Date and start time of the event.
Location	NVARCHAR(150)		Town/venue, used for the browse-events list.
Description	NVARCHAR(1000)		Free-text details shown to participants.

4. Categories

A specific entry option within an event, such as a distance or price tier (e.g. “5km Fun Run”, “40km Road Ride”). Categories, not Events directly, is what a participant actually enrols into – this lets one event offer several distances/prices without duplicating event-level data.

Attribute	Type (SQL)	Key	Notes
CategoryID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
EventID	INT	**FK**	References Events.EventID – the event this category belongs to.
CategoryName	NVARCHAR(100)		e.g. “10km Race”.
Distance	DECIMAL(6,2)		Distance in km.
MaxParticipants	INT		Capacity cap; defaults to 100.
Price	DECIMAL(8,2)		Entry fee; defaults to 0 for free events.

5. Enrolments

The associative (junction) entity that resolves the many-to-many relationship between Participants and Categories – one participant can enter many categories over time, and one category can be entered by many participants. Modelling this as its own table (rather than a direct many-to-many link) is what lets the system store enrolment-specific data such as the date entered and status, and is exactly what the Results table hangs off in turn.

Attribute	Type (SQL)	Key	Notes
EnrolmentID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
ParticipantID	INT	**FK**	References Participants.ParticipantID.
CategoryID	INT	**FK**	References Categories.CategoryID.
EnrolmentDate	DATETIME		Defaults to GETDATE().
Status	NVARCHAR(20)		Confirmed / Cancelled; defaults to 'Confirmed'.

6. Results

The finish-time record captured by an organiser once a participant has completed their category. Linked one-to-one with Enrolments (an enrolment produces at most one result) and also references the organiser who captured it, so the system can audit who recorded each result.

Attribute	Type (SQL)	Key	Notes
ResultID	INT IDENTITY	**PK**	Auto-incrementing surrogate key.
EnrolmentID	INT	**FK (UNIQUE)**	References Enrolments.EnrolmentID – UNIQUE enforces the 1:1 relationship.
RecordedByOrganiserID	INT	**FK**	References Organisers.OrganiserID.
FinishTime	TIME		Race finish time.
Position	INT		Overall or category placing; nullable until ranking is finalised.
RecordedAt	DATETIME		Defaults to GETDATE().

Normalization

The schema is in Third Normal Form (3NF). Every table has a single-column surrogate primary key, so all non-key attributes depend on the whole key (2NF is satisfied trivially). There are no transitive dependencies between non-key attributes – for example, Events does not repeat OrganiserName or OrganiserEmail alongside OrganiserID; that data lives only in Organisers and is reached via the foreign key (3NF). The one deliberate case that looks like repeated data – Results storing RecordedByOrganiserID as well as Enrolments already linking to a Category/Participant – is not a normalization violation: RecordedByOrganiserID answers a different question (who captured the result) to EnrolmentID (which entry the result belongs to), and neither can be derived from the other.

Relationship-by-relationship reasoning

- Organisers → Events (1:M): an organiser account can create and manage many events over time, but each event has exactly one owning organiser – so the foreign key (OrganiserID) sits on the "many" side, in Events.
- Events → Categories (1:M): each event can offer several categories (5km, 10km, etc.), but every category belongs to exactly one event – the foreign key (EventID) sits in Categories.
- Participants ↔ Categories, resolved via Enrolments (M:M → 1:M + 1:M): a participant enters many categories across different events, and a category is entered by many participants. This many-to-many is resolved into two one-to-many relationships – Participants → Enrolments and Categories → Enrolments – with Enrolments holding both foreign keys.
- Enrolments → Results (1:1): a single enrolment produces, at most, one finish-time result. This is enforced in the schema with a UNIQUE constraint on Results.EnrolmentID, so the same enrolment can never be linked to two different results.

- Organisers → Results (1:M): one organiser can capture results for many different enrolments across the events they run, but each result was captured by exactly one organiser – tracked via RecordedByOrganiserID for accountability/auditing.

Relationship summary table

Relationship	Cardinality	Description
Organisers → Events	1 : M	One organiser creates many events.
Events → Categories	1 : M	One event has many categories (e.g. 5km, 10km).
Participants → Enrolments	1 : M	One participant can make many enrolments.
Categories → Enrolments	1 : M	One category can receive many enrolments.
Enrolments → Results	1 : 1	Each enrolment produces at most one result.
Organisers → Results	1 : M	One organiser can capture many results.

Section B – API Endpoint Plan

The table below plans every API endpoint the RaceDay system will expose in Part 2, covering Authentication, User Profile, Events, Categories, Event Enrolments and Results. “Any” under Role Required means any authenticated user (Organiser or Participant); “None” means the endpoint is public.

Method	Route	Description	Role	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either an Organiser or a Participant.	None	{ fullName, email, password, role }	201 Created – user created. 400 Bad Request – invalid/duplicate email.
POST	/api/auth/login	Authenticates a user and returns a JWT access token.	None	{ email, password }	200 OK – token + role. 401 Unauthorised – bad credentials.
GET	/api/users/me	Returns the profile of the currently logged-in user.	Any	None	200 OK – user profile object.
PUT	/api/users/me	Updates the profile of the currently logged-in user.	Any	{ fullName, phoneNumber, dateOfBirth }	200 OK – updated profile. 400 Bad Request – validation error.
GET	/api/events	Lists all upcoming events, optionally filtered by type or date.	None	None	200 OK – array of events.
GET	/api/events/{id}	Returns full details for a single event, including its categories.	None	None	200 OK – event object. 404 Not Found.
POST	/api/events	Creates a new event owned by the logged-in organiser.	Organiser	{ eventName, eventType, eventDate, location, description }	201 Created – new event. 400 Bad Request.
PUT	/api/events/{id}	Updates an event owned by the logged-in organiser.	Organiser	{ eventName, eventType, eventDate, location, description }	200 OK – updated event. 404 Not Found. 403 Forbidden – not the owner.
DELETE	/api/events/{id}	Deletes an event owned by the logged-in organiser.	Organiser	None	204 No Content. 404 Not Found. 403 Forbidden.
GET	/api/events/{eventId}/categories	Lists all categories for a specific event.	None	None	200 OK – array of categories.
POST	/api/events/{eventId}/categories	Adds a new category to an event.	Organiser	{ categoryName, distance, maxParticipants, price }	201 Created – new category. 404 Not Found – event does not exist.
PUT	/api/categories/{id}	Updates an existing category.	Organiser	{ categoryName, distance, maxParticipants, price }	200 OK – updated category. 404 Not Found.
DELETE	/api/categories/{id}	Deletes a category.	Organiser	None	204 No Content. 404 Not Found.
POST	/api/categories/{categoryId}/enrolments	Enrols the logged-in participant into a category.	Participant	{ } (participant identified from the JWT token)	201 Created – enrolment record. 409 Conflict – already enrolled. 400 Bad Request – category full.
GET	/api/enrolments/me	Returns all enrolments (past and upcoming) for the logged-in participant.	Participant	None	200 OK – array of enrolments.
GET	/api/events/{eventId}/enrolments	Lists all participant enrolments for an event.	Organiser	None	200 OK – array of enrolments. 403 Forbidden – not the event owner.
POST	/api/enrolments/{enrolmentId}/results	Captures the finish time/position for an enrolment.	Organiser	{ finishTime, position }	201 Created – result recorded. 404 Not Found – enrolment does not exist. 409 Conflict – result already captured.

GET	/api/results/me	Returns the logged-in participant's personal result history.	Participant	None	200 OK – array of results.
GET	/api/events/{eventId}/results	Returns all captured results for an event.	None	None	200 OK – array of results, ordered by position.

Section C – SQL Database Script

The script below creates the RaceDayDB database in SQL Server Management Studio (SSMS), matching the ERD in Section A exactly. It defines all six tables with primary keys, foreign keys and constraints (NOT NULL, UNIQUE, DEFAULT, plus CHECK constraints on EventType, Status and numeric ranges), adds indexes on every foreign key column to keep joins and lookups efficient, then seeds the database with two organisers, two participants, three events, categories for each event, and sample enrolments and results. The full script is also committed as docs/raceday_database.sql in the GitHub repository. Test it on a clean SQL Server instance before submitting.

```
/* ============================================================
   RaceDay - Database Creation Script
   System : RaceDay Event Management System
   Author : Tim
   Target : Microsoft SQL Server (SSMS)
   Notes  : Run on a clean SQL Server instance. Creates the
            RaceDayDB database, all tables with keys and
            constraints, then seeds sample data.
   ============================================================ */

IF DB_ID('RaceDayDB') IS NOT NULL
BEGIN
    ALTER DATABASE RaceDayDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE RaceDayDB;
END
GO

CREATE DATABASE RaceDayDB;
GO

USE RaceDayDB;
GO

/* ------------------------------------------------------------
   1. Organisers
   ------------------------------------------------------------ */
CREATE TABLE Organisers (
    OrganiserID     INT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(100)   NOT NULL,
    Email           NVARCHAR(150)   NOT NULL UNIQUE,
    PasswordHash    NVARCHAR(255)   NOT NULL,
    PhoneNumber     NVARCHAR(20)    NULL,
    CreatedAt       DATETIME        NOT NULL DEFAULT GETDATE()
);
GO

/* ------------------------------------------------------------
   2. Participants
   ------------------------------------------------------------ */
CREATE TABLE Participants (
    ParticipantID   INT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(100)   NOT NULL,
    Email           NVARCHAR(150)   NOT NULL UNIQUE,
    PasswordHash    NVARCHAR(255)   NOT NULL,
    DateOfBirth     DATE            NULL,
    PhoneNumber     NVARCHAR(20)    NULL,
    CreatedAt       DATETIME        NOT NULL DEFAULT GETDATE()
);
GO

/* ------------------------------------------------------------
   3. Events
   ------------------------------------------------------------ */
CREATE TABLE Events (
    EventID         INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserID     INT             NOT NULL,
    EventName       NVARCHAR(150)   NOT NULL,
    EventType       NVARCHAR(50)    NOT NULL,   -- Running / Walking / Cycling
    EventDate       DATETIME        NOT NULL,
    Location        NVARCHAR(150)   NOT NULL,
    Description     NVARCHAR(1000)  NULL,
    CreatedAt       DATETIME        NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Organisers FOREIGN KEY (OrganiserID)
        REFERENCES Organisers (OrganiserID),
    CONSTRAINT CK_Events_EventType CHECK (EventType IN ('Running', 'Walking', 'Cycling'))
);
```

```
GO

CREATE INDEX IX_Events_OrganiserID ON Events (OrganiserID);
GO

/* ------------------------------------------------------------
   4. Categories
   ------------------------------------------------------------ */
CREATE TABLE Categories (
    CategoryID      INT IDENTITY(1,1) PRIMARY KEY,
    EventID         INT             NOT NULL,
    CategoryName    NVARCHAR(100)   NOT NULL,
    Distance        DECIMAL(6,2)    NOT NULL,   -- km
    MaxParticipants INT             NOT NULL DEFAULT 100,
    Price           DECIMAL(8,2)    NOT NULL DEFAULT 0,
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventID)
        REFERENCES Events (EventID),
    CONSTRAINT CK_Categories_Distance CHECK (Distance > 0),
    CONSTRAINT CK_Categories_MaxParticipants CHECK (MaxParticipants > 0)
);
GO

CREATE INDEX IX_Categories_EventID ON Categories (EventID);
GO

/* ------------------------------------------------------------
   5. Enrolments (resolves the Participants <-> Categories
      many-to-many relationship)
   ------------------------------------------------------------ */
CREATE TABLE Enrolments (
    EnrolmentID     INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantID   INT             NOT NULL,
    CategoryID      INT             NOT NULL,
    EnrolmentDate   DATETIME        NOT NULL DEFAULT GETDATE(),
    Status          NVARCHAR(20)    NOT NULL DEFAULT 'Confirmed',   -- Confirmed / Cancelled
    CONSTRAINT FK_Enrolments_Participants FOREIGN KEY (ParticipantID)
        REFERENCES Participants (ParticipantID),
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryID)
        REFERENCES Categories (CategoryID),
    CONSTRAINT UQ_Enrolments_Participant_Category UNIQUE (ParticipantID, CategoryID),
    CONSTRAINT CK_Enrolments_Status CHECK (Status IN ('Confirmed', 'Cancelled'))
);
GO

CREATE INDEX IX_Enrolments_ParticipantID ON Enrolments (ParticipantID);
CREATE INDEX IX_Enrolments_CategoryID ON Enrolments (CategoryID);
GO

/* ------------------------------------------------------------
   6. Results
   ------------------------------------------------------------ */
CREATE TABLE Results (
    ResultID                INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentID             INT             NOT NULL UNIQUE,
    RecordedByOrganiserID   INT             NOT NULL,
    FinishTime              TIME            NOT NULL,
    Position                INT             NULL,
    RecordedAt              DATETIME        NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentID)
        REFERENCES Enrolments (EnrolmentID),
    CONSTRAINT FK_Results_Organisers FOREIGN KEY (RecordedByOrganiserID)
        REFERENCES Organisers (OrganiserID),
    CONSTRAINT CK_Results_Position CHECK (Position IS NULL OR Position > 0)
);
GO

CREATE INDEX IX_Results_RecordedByOrganiserID ON Results (RecordedByOrganiserID);
GO

/* ============================================================
   Sample data
   ============================================================ */

-- Organisers (2)
INSERT INTO Organisers (FullName, Email, PasswordHash, PhoneNumber) VALUES
('Sipho Ndlovu', 'sipho.ndlovu@raceday.co.za', 'HASHED_PWD_1', '0821234567'),
```

```
('Anke van der Merwe', 'anke.vdm@raceday.co.za', 'HASHED_PWD_2', '0837654321');

-- Participants (2)
INSERT INTO Participants (FullName, Email, PasswordHash, DateOfBirth, PhoneNumber) VALUES
('Thabo Mokoena', 'thabo.mokoena@example.com', 'HASHED_PWD_3', '1994-03-12', '0731122334'),
('Lindiwe Zulu', 'lindiwe.zulu@example.com', 'HASHED_PWD_4', '1998-07-25', '0729988776');

-- Events (3)
INSERT INTO Events (OrganiserID, EventName, EventType, EventDate, Location, Description) VALUES
(1, 'Vryheid Community Fun Run', 'Running', '2026-10-10 07:00:00', 'Vryheid, KwaZulu-Natal', 'Annual community
fun run supporting local schools.'),
(1, 'Vryheid Cycle Challenge', 'Cycling', '2026-11-14 06:30:00', 'Vryheid, KwaZulu-Natal', 'Road cycling event
over three route distances.'),
(2, 'Durban Beachfront Park Walk', 'Walking', '2026-09-20 08:00:00', 'Durban, KwaZulu-Natal', 'Family-friendly
walk along the Durban beachfront.');

-- Categories (2 or more per event)
INSERT INTO Categories (EventID, CategoryName, Distance, MaxParticipants, Price) VALUES
(1, '5km Fun Run', 5.00, 200, 80.00),
(1, '10km Race', 10.00, 150, 120.00),
(2, '40km Road Ride', 40.00, 100, 200.00),
(2, '80km Road Ride', 80.00, 80, 250.00),
(3, '3km Family Walk', 3.00, 300, 0.00);

-- Enrolments (sample)
INSERT INTO Enrolments (ParticipantID, CategoryID, Status) VALUES
(1, 1, 'Confirmed'),
(1, 3, 'Confirmed'),
(2, 2, 'Confirmed'),
(2, 5, 'Confirmed');

-- Results (sample, captured by an organiser)
INSERT INTO Results (EnrolmentID, RecordedByOrganiserID, FinishTime, Position) VALUES
(1, 1, '00:28:14', 12),
(3, 1, '00:52:03', 5);
GO
```

References

The following sources were consulted while planning the ERD, the API endpoints, and the SQL database script for this part of the POE.

- Microsoft Learn. CREATE TABLE (Transact-SQL). https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql (accessed August 2026).
- Microsoft Learn. SQL Server Data Types. https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql (accessed August 2026).
- Fowler, M. Patterns of Enterprise Application Architecture – reference for the Enrolments associative-entity pattern used to resolve the Participants–Categories many-to-many relationship.
- Microsoft Learn. RESTful web API design. https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design (accessed August 2026) – guidance on resource naming, HTTP verbs and status codes used in Section B.
- Mozilla Developer Network. HTTP response status codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status (accessed August 2026).
- Rosebank College. RaceDay Portfolio of Evidence assignment brief, 2026 – source of all functional requirements planned in this document.
- Anthropic. Claude (AI assistant) – used to help structure the ERD, draft the endpoint plan table, and produce this document from the requirements above; all design decisions were reviewed and adjusted by the author.